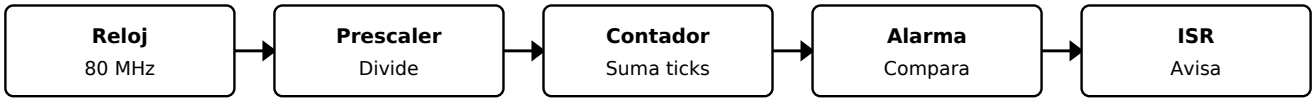
Resumen de la semana 3

Temporizadores de hardware en el ESP32

1. Para qué sirve un temporizador

Un **temporizador de hardware** es un circuito dentro del microcontrolador que cuenta pulsos de reloj. Cada pulso representa una cantidad conocida de tiempo. El circuito sigue contando sin que `loop()` tenga que consultar constantemente.

La diferencia es importante: `delay()` detiene la ejecución del programa, `millis()` obliga a comprobar el reloj en cada vuelta y un **timer** puede avisar mediante una interrupción justo al alcanzar el valor configurado.



2. Reloj, prescaler, resolución y alarma

El reloj de entrada de los temporizadores del ESP32 explicado en clase es de **80 MHz**, es decir, 80 000 000 pulsos por segundo. El **prescaler** divide esa frecuencia para que el contador avance a un ritmo más fácil de usar.

Concepto	Fórmula o significado
Frecuencia de conteo	`f = 80 000 000 / N`, donde N es el prescaler.
Resolución	`resolución = 1 / f`, duración de cada tick.
Tiempo	`t = alarma × resolución`.
Valor de alarma	`alarma = t × f`.

Con `N = 80`, la frecuencia es `f = 80 000 000 / 80 = 1 000 000` ticks/s. Por lo tanto, la **resolución es de 1 µs**. Así, una alarma de 500 000 cuentas equivale a medio segundo.

3. Ejemplos de cálculo paso a paso

Ejemplo 1: aviso cada 0,5 segundos

Datos: reloj de 80 MHz, `f = 1 000 000` ticks/s y tiempo deseado `t = 0,5 s`.

- Prescaler: `N = 80 000 000 / 1 000 000 = 80`.
- Resolución: `1 / 1 000 000 = 0,000001 s = 1 µs`.
- Alarma: `0,5 × 1 000 000 = 500 000` cuentas.
- Comprobación: `500 000 × 1 µs = 0,5 s`.

Ejemplo 2: frecuencia de 10 000 ticks/s

Si `f = 10 000` y la alarma vale 2 500, entonces `resolución = 1/10 000 = 100 µs` y `t = 2 500 × 100 µs = 0,25 s`. Son **4 avisos por segundo**. El prescaler requerido es `80 000 000 / 10 000 = 8 000`.

Actividad de clase: tres cálculos

Caso	Cálculo	Respuesta
A. `f = 1 MHz`, alarma `2 000`	`t = 2 000 / 1 000 000`	**0,002 s = 2 ms**
B. Aviso cada `10 ms`, `f = 1 MHz`	`alarma = 0,010 × 1 000 000`	**10 000 cuentas**
C. `f = 40 000`, alarma `40 000`	`t = 40 000 / 40 000`; `N = 80 000 000 / 40 000`	**1 s**; prescaler **2 000**

4. Autorrecarga, límite del contador y tipos de timer

La **alarma** es el número que se compara con el contador. Si está activa la **autorrecarga**, al llegar a ese valor se genera el evento y el contador vuelve a empezar automáticamente. El aviso se repite con el mismo intervalo.

Desbordamiento es distinto: ocurre al superar el mayor número que permite el contador, $2^n - 1$. La alarma con autorrecarga se establece por decisión del programador; el desbordamiento depende del tamaño físico del registro.

Contador	Mayor valor posible	Con ticks de 1 µs
8 bits	255	Unos 0,26 ms
16 bits	65 535	Unos 65,5 ms
64 bits	18 446 744 073 709 551 615	Muy superior a la duración habitual de una aplicación

El ESP32-WROOM-32 tiene **4 temporizadores de propósito general**, repartidos entre TIMG0 y TIMG1, con contadores de 64 bits y prescaler. El ATmega328P tiene **3 temporizadores**, de 8 o 16 bits.

5. De la alarma a la interrupción

La CPU configura el timer a través de registros o de una **API**. El **core de Espressif** permite usar el ESP32 desde Arduino IDE; la versión 3.x cambió nombres y parámetros de funciones respecto de la 2.x. Por eso hay que revisar para qué versión se escribió un ejemplo.

Cuando la alarma se cumple, el temporizador genera una **interrupción**. La CPU pausa temporalmente el trabajo normal, ejecuta una rutina de atención o **ISR** y después continúa. Una función **callback** es una función escrita por el programador que el sistema llama cuando sucede un evento.

La ISR debe durar muy poco. `IRAM_ATTR` permite colocarla en memoria de instrucciones interna; `volatile` se usa para una variable compartida con el programa principal. La idea es que la ISR solamente cambie una bandera y que `loop()` haga el trabajo que requiere más tiempo.

```
volatile bool aviso = false;

void IRAM_ATTR atenderTimer() {
  aviso = true;          // trabajo rapido en la ISR
}

void loop() {
  if (aviso) {
    aviso = false;
    // procesar el evento en el programa principal
  }
}
```

Este fragmento resume el **patrón bandera** visto en clase; no reemplaza la configuración de un temporizador. La interrupción externa de un pulsador se trabaja en la semana 4.

6. Repaso corto

¿Qué significa una resolución de 100 µs? Que el contador avanza una unidad cada 100 microsegundos; no puede distinguir cambios menores con esa configuración.

Si la alarma está en 500 000 con `f = 1 MHz`**, ¿qué pasa?** Se produce un aviso cada **0,5 segundos**, es decir, **2 por segundo**, si el timer está en autorrecarga.

¿Para qué sirve `static`**?** Para que una variable local conserve su valor entre distintas ejecuciones de una función.

¿Qué pasa con `uint8_t r = 200 + 100`**?** El resultado matemático es 300, pero en un entero sin signo de 8 bits queda **44** al almacenarse.

> **Lo imprescindible:** saber explicar de dónde salen el prescaler, la resolución y el valor de alarma. En los ejercicios conviene escribir primero la fórmula, sustituir los datos y convertir correctamente segundos, milisegundos y microsegundos.